Companies generate large amounts of data every day. Examples include sales transactions, customer orders, bank payments, manufacturing output, website visits, and hospital records.

Raw data by itself is not always useful. Companies need to process it, calculate meaningful metrics, identify patterns, and present the results in a form that people can understand.

Python provides several libraries for this work:

NumPy — Numerical computing

Used for numerical arrays, mathematical calculations, statistical operations, and efficient calculations across large datasets.

Pandas — Data manipulation and analysis

Used to import data from CSV or Excel, clean missing values, remove duplicates, filter records, group data, and summarize business performance.

Matplotlib — Data visualization

Used to create charts and graphs that help analysts, managers, and business teams understand trends, comparisons, and unusual changes.

Import the libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
print("Libraries imported successfully!")
```

```
NumPy version: 2.1.3
Pandas version: 2.2.3
Libraries imported successfully!
```

Numpy:

Create a NumPy array

```
daily_units = np.array([5, 7, 4, 8, 6])

print("Daily laptop units:", daily_units)
print("Data type:", daily_units.dtype)
print("Number of values:", daily_units.size)
```

```
Daily laptop units: [5 7 4 8 6]
Data type: int64
Number of values: 5
```

Calculate sales statistics with NumPy

```
print("Total units sold:", np.sum(daily_units))
print("Minimum daily sales:", np.min(daily_units))
print("Maximum daily sales:", np.max(daily_units))
print("Average daily sales:", np.mean(daily_units))
print("Median daily sales:", np.median(daily_units))
print("Standard deviation:", round(np.std(daily_units), 2))
```

```
Total units sold: 30
Minimum daily sales: 4
Maximum daily sales: 8
Average daily sales: 6.0
Median daily sales: 6.0
Standard deviation: 1.41
```

Calculate revenue with NumPy

```
laptop_price = 40000

daily_revenue = daily_units * laptop_price

print("Daily revenue:", daily_revenue)
print("Total revenue:", np.sum(daily_revenue))
print("Average daily revenue:", np.mean(daily_revenue))
```

```
Daily revenue: [200000 280000 160000 320000 240000]
Total revenue: 1200000
Average daily revenue: 240000.0
```

Pandas

Create the company DataFrame

```
data = {
    "Day": [
        "Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
        "Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
        "Monday", "Tuesday", "Wednesday", "Thursday", "Friday"
    ],
    "Product": ["Laptop"] * 5 + ["Mouse"] * 5 + ["Keyboard"] * 5,
    "Units_Sold": [
        5, 7, 4, 8, 6,
        20, 25, 18, 30, 22,
        10, 12, 9, 15, 11
    ],
    "Price": [40000] * 5 + [500] * 5 + [1500] * 5
}

df = pd.DataFrame(data)

print(df)
```

	Day	Product	Units_Sold	Price
0	Monday	Laptop	5	40000
1	Tuesday	Laptop	7	40000
2	Wednesday	Laptop	4	40000
3	Thursday	Laptop	8	40000
4	Friday	Laptop	6	40000
5	Monday	Mouse	20	500
6	Tuesday	Mouse	25	500
7	Wednesday	Mouse	18	500
8	Thursday	Mouse	30	500
9	Friday	Mouse	22	500
10	Monday	Keyboard	10	1500
11	Tuesday	Keyboard	12	1500
12	Wednesday	Keyboard	9	1500
13	Thursday	Keyboard	15	1500
14	Friday	Keyboard	11	1500

Inspect the dataset

```
df.head()
```

	Day	Product	Units_Sold	Price
0	Monday	Laptop	5	40000
1	Tuesday	Laptop	7	40000
2	Wednesday	Laptop	4	40000
3	Thursday	Laptop	8	40000
4	Friday	Laptop	6	40000

```
df.tail()
```

	Day	Product	Units_Sold	Price
10	Monday	Keyboard	10	1500
11	Tuesday	Keyboard	12	1500
12	Wednesday	Keyboard	9	1500
13	Thursday	Keyboard	15	1500
14	Friday	Keyboard	11	1500

```
df.columns
```

```
Index(['Day', 'Product', 'Units_Sold', 'Price'], dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Day         15 non-null     object
 1   Product     15 non-null     object
 2   Units_Sold  15 non-null     int64 
 3   Price       15 non-null     int64 
dtypes: int64(2), object(2)
memory usage: 612.0+ bytes
```

Add a revenue column

```
df["Revenue"] = df["Units_Sold"] * df["Price"]

print(df[["Day", "Product", "Units_Sold", "Price", "Revenue"]])
```

```
           Day   Product  Units_Sold  Price  Revenue
0       Monday    Laptop           5  40000   200000
1      Tuesday    Laptop           7  40000   280000
2    Wednesday    Laptop           4  40000   160000
3     Thursday    Laptop           8  40000   320000
4       Friday    Laptop           6  40000   240000
5       Monday     Mouse          20    500    10000
6      Tuesday     Mouse          25    500    12500
7    Wednesday     Mouse          18    500     9000
8     Thursday     Mouse          30    500    15000
9       Friday     Mouse          22    500    11000
10      Monday  Keyboard          10   1500    15000
11     Tuesday  Keyboard          12   1500    18000
12   Wednesday  Keyboard           9   1500    13500
13    Thursday  Keyboard          15   1500    22500
14      Friday  Keyboard          11   1500    16500
```

Find revenue by product

Check data quality

```
print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())
```

```
Missing values:
Day           0
Product       0
Units_Sold    0
Price         0
Revenue       0
dtype: int64

Duplicate rows:
0
```

Matplotlib:

```
daily_units = df.groupby("Day")["Units_Sold"].sum()

day_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
daily_units = daily_units.reindex(day_order)

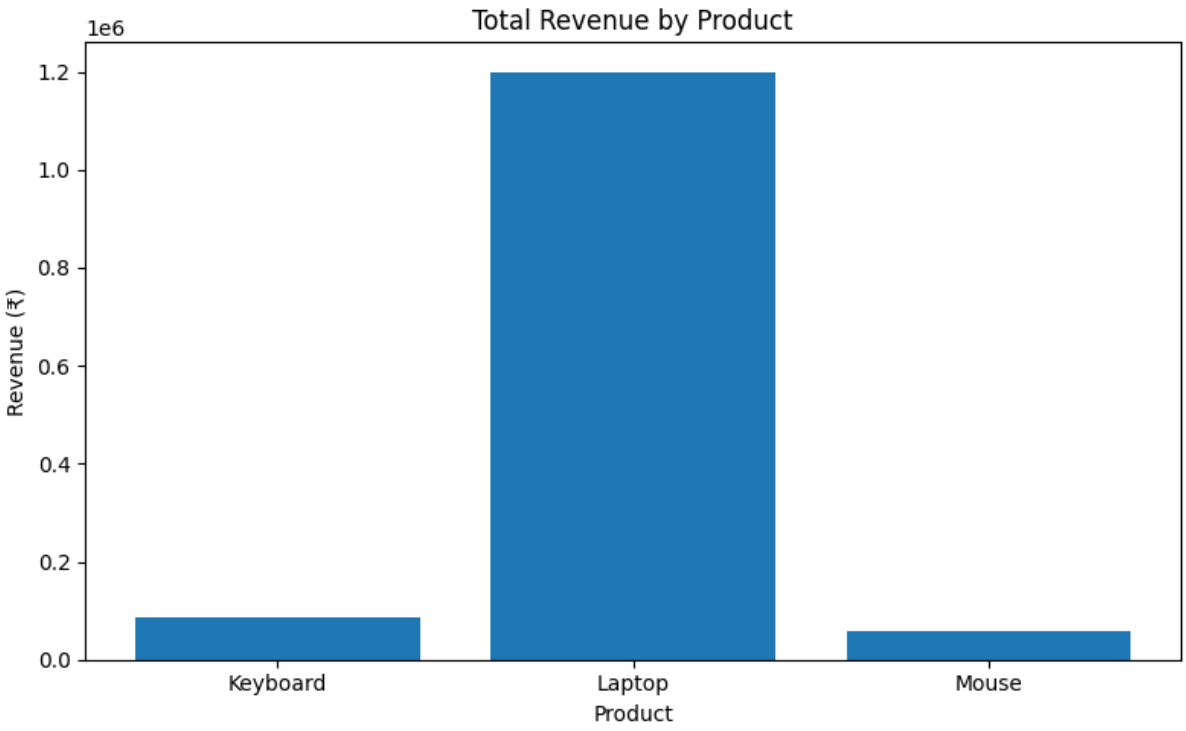
plt.figure(figsize=(8, 5))
plt.plot(daily_units.index, daily_units.values, marker="o")

plt.title("Daily Units Sold")
plt.xlabel("Day")
plt.ylabel("Total Units Sold")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
revenue_by_product = df.groupby("Product")["Revenue"].sum()

plt.figure(figsize=(8, 5))
plt.bar(revenue_by_product.index, revenue_by_product.values)

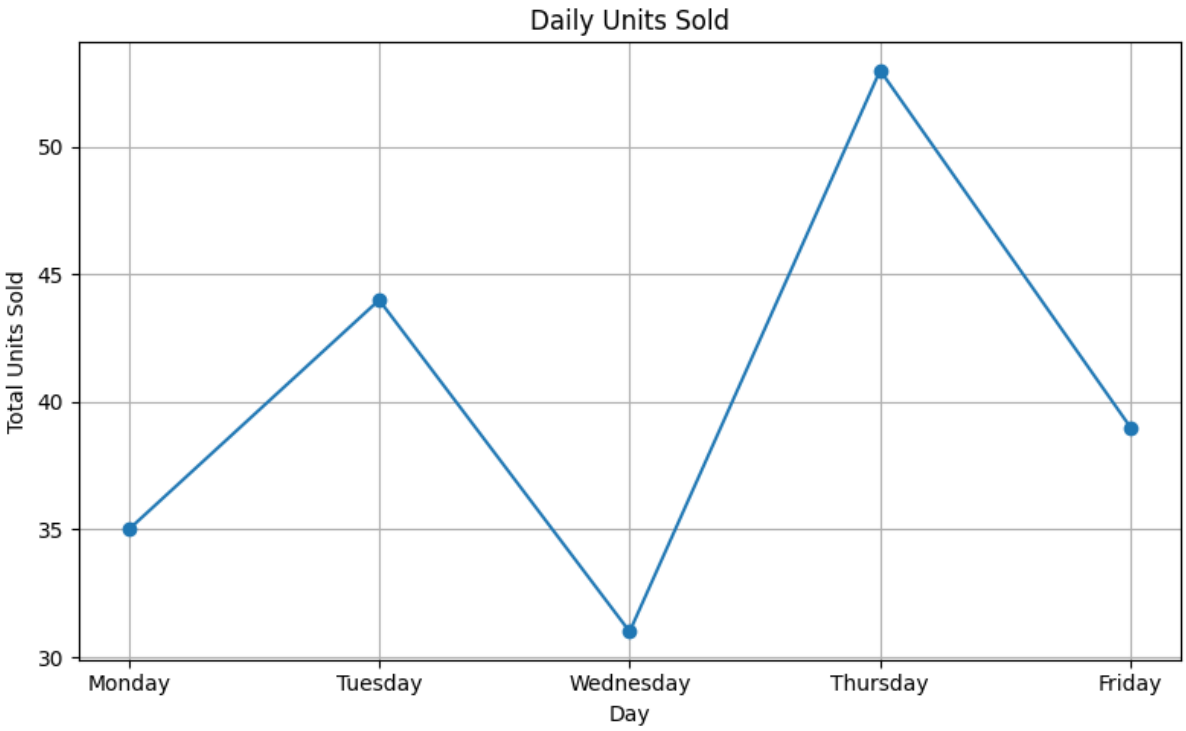
plt.title("Total Revenue by Product")
plt.xlabel("Product")
plt.ylabel("Revenue (₹)")
plt.tight_layout()
plt.show()
```




```
revenue_by_product = df.groupby("Product")["Revenue"].sum()

print(revenue_by_product)
```

```
Product
Keyboard      85500
Laptop      1200000
Mouse         57500
Name: Revenue, dtype: int64
```


Daily Units Sold
Total Units Sold
50
45
40
35
30
Monday
Tuesday
Wednesday
Thursday
Friday
Day